

Ventago — 아키텍처 & 개발 규약 핸드북

다점포 소매업 POS/ERP 시스템 · 재고·판매·재무·생산·외주 종합 업무 플랫폼 · CLAUDE.md 기준 재작성

1 프로젝트 개요

다점포 소매업 대상 POS/ERP. 재고·판매·재무·생산·외주 관리를 포함한 종합 업무 플랫폼.

운영 API	<code>https://newapi.coolsistema.com/api</code>
운영 프론트	<code>https://ventago.coolsistema.com</code>
운영 서버	srv803182 · IP 62.72.7.245 · 포트 5002
배포	Jenkins CI/CD → Docker
저장소	npm workspaces 모노레포

저장소 구조

```
ACE_online_1.0/
├─ api-ventago/    # NestJS 백엔드 (포트 5002)
├─ ventago-app/    # Next.js 프론트 (5001 docker / 3050 dev)
├─ print-agent/    # 열감지 (comandera) 프린터 (Electron)
└─ zebra-agent/    # Zebra 바코드 라벨 (Electron)
```

2 기술 스택

4개 서브프로젝트의 프레임워크·라이브러리 구성.



백엔드

api-ventago

Framework	NestJS 11 + TS
ORM	Sequelize (underscored)
DB	PostgreSQL 15 (ventago)
인증	JWT (Passport)
파일	MinIO (S3 호환)
실시간	Socket.io
로그인	Winston
스케줄	@nestjs/schedule



프론트

ventago-app

Framework	Next.js 13 (Pages)
React	React 18
UI	MUI 5
상태	Redux Toolkit
폼	React Hook Form + Yup
HTTP	Axios (apiConnector)
캐시	SWR (5분 dedup)
인가	CASL · 분석 PostHog



에이전트

print · zebra

Framework	Electron 28
WS	socket.io-client
네임스페이스	/print-agent
인증	API Key (BranchAgent)
print	ESC/POS (Net+USB)
zebra	ZPL II (TCP 9100)
라벨	50x25 simple/doble · 100x25
테마	#1a1a2e + #f5a623

서버 URL 고정: 에이전트는 서버 URL 입력이 불필요합니다 — 운영 `http://62.72.7.245:5002/api`, 개발 `http://localhost:5002/api` 가 코드에 고정되어 있고 사용자는 **API Key**만 입력합니다.

3 멀티테넌트 구조 & DB 규칙

거의 모든 테이블에 `store_id` FK가 있어 매장 단위로 데이터가 격리됩니다.

계층 구조 (1:N:N:N)



- Branch 생성 시 기본 Box + Terminal 자동 생성 (`branch.service.ts` → `createBranch`)
- 매장 최초 등록 시 기본 Branch/Box/Terminal 생성 (`storeTemplate.service.ts` → `createStoreDefaults`)

DB 컬럼 네이밍

Sequelize `underscored: true` 전역 설정으로 모델 camelCase → DB snake_case 자동 매핑.

모델 (CAMELCASE)	DB (SNAKE_CASE)
logoUrl	logo_url
aliasName	alias_name

SQL 직접 실행 시 반드시 `snake_case` 사용.

스키마 reference (추측 금지)

SQL/마이그레이션/raw query 작성 전 반드시 참조:

`.planning/intel/db-schema-tables.md`

133개 테이블의 모든 컬럼 (타입/NOT NULL/default)

`.planning/intel/db-schema-fks.md`

모든 외래 키 관계

```
# 스키마 변경 후 재생성
./planning/intel/db-schema.regen.sh
```

자주 헷갈리는 컬럼명 (실수 방지):

- sales 테이블은 `branch_id` 없음 — 지점은 `user_id` → `users.branch_id` 경유
- `terminal_id` 만 직접 FK. box/branch 도달은 join 필요
- `sale_items` 의 promo 컬럼: `is_promo_free / promotion_id / promo_group_id`

4 아키텍처 핵심 규칙

프린터 에이전트 · MinIO · API 서비스 · 인증 컨텍스트의 공통 규약.

프린터 에이전트 구조

```
branch_agents (지점당 N개 등록 가능)
  branchId      # NOT UNIQUE → 다중 프린터
  agentType     # 'thermal' | 'zebra'
  label         # 'Comandera Cocina' 등
  apiKey        # UNIQUE (에이전트별 고유)
  isOnline, socketId, lastSeenAt
```

```
terminals (터미널별 에이전트 매핑)
  thermalAgentId FK → branch_agents
  zebraAgentId   FK → branch_agents
```

파일 업로드 (MinIO)

- MinioModule 을 해당 모듈 imports 에 추가
- MinioService.uploadFile(file, fileName) → { fileName } 반환
- 프론트 이미지 URL: {API_HOST}/minio/{fileName}

API 서비스 `api.service.ts`

```
apiConnector.get(path)
apiConnector.post(path, body)
apiConnector.put(path, body)
apiConnector.remove(path)          # DELETE
apiConnector.sendFile(path, form) # POST multipart
apiConnector.putFile(path, form)  # PUT multipart
```

인증 컨텍스트

- useAuth() 혹은 user 접근 — storeId · storeName · aliasName · logoUrl + 권한 정보 포함
- AuthContext + BranchContext 분리: selectedBranchId 는 BranchContext 에서 관리 → 지점 전환 시 **110+ 컴포넌트 불필요 리렌더 방지**
- auth 응답은 auth.service.ts 의 /me 엔드포인트에서 구성

5 세션 & 터미널 보안

api-ventago/src/app/session/ — 중복 로그인 절대 차단 + 디바이스/IP 기반 부정 사용 방지.

active_sessions

유저당 1개만 존재 (UNIQUE userId). 새 로그인 시 기존 세션 삭제 → 기존 세션 즉시 **401 SESSION_EXPIRED**

terminal_devices

브라우저 fingerprint ↔ 터미널 바인딩. 새 디바이스 접속 시 터미널 등록 강제

branch_ip_registries

public IP ↔ 지점(sucursal) 매핑. 새 IP 접속 시 지점 등록 강제

로그인 플로우

1. 자격 증명 검증

기존 ActiveSession 삭제
(중복 로그인 차단)



2. IP 확인

미등록 IP →
requireBranchRegistration:
true



3. Fingerprint 확인

미등록 →
requireTerminalRegistration:
true



4. sessionToken 발급

정상이면 UUID v4 발급

프론트엔드 연동: api.service.ts 모든 요청에 x-session-token 헤더 자동 주입 · AuthContext.tsx 디바이스 fingerprint 수집 · device-fingerprint.ts 브라우저 특성 SHA-256 해시 · **SessionGuard** — JWT 인증 후 추가로 sessionToken 검증 (@UseGuards(SessionGuard))

6 모듈 & 페이지 맵

백엔드 모듈(api-ventago/src/app/)과 프론트 페이지(ventago-app/src/pages/).

백엔드 모듈

auth · JWT 인증/가드 store · 매장(로고) branch · 지점 users · 사용자/역할/권한 products · 상품/재고
sales · 판매/결제/할인 expenses · 비용 box · 금전함 caja-fuerte · 금고 production · BOM/작업지시
subcon · 외주 print · 프린터 에이전트 terminal · 터미널 marketplace revendedor · 재판매 session · 세션 보안
chat · AI 채팅

프론트 페이지

nueva-venta · POS ventas · 판매내역 productos precios · 가격 gastos · 비용 caja caja-fuerte
control-de-caja sucursales · 지점 usuarios talleres · 외주 dashboards reportes · 보고서 configuracion · 설정
herramientas · 다운로드 admin · 관리자

sucursales/[id]/impresora — 에이전트(프린터) 관리 페이지

레이아웃 & 사이드바 리렌더 방지

- 진입점 layouts/UserLayout.tsx → 프레임 @core/layouts/VerticalLayout.tsx → 네비 .../navigation/index.tsx (React.memo)
- Navigation: **React.memo** 적용, darkTheme 을 useMemo 로 메모이제이션
- UserLayout: navMenuBranding · appBarContent 등을 useCallback/useMemo 로 추출
- VerticalLayout: {...props} 스프레드 제거하고 명시적 props 전달

7 성능 최적화 규약

300ms 타겟 — 모든 사이드바 메뉴 클릭 → 콘텐츠 렌더 완료까지 P95 ≤ 300ms.

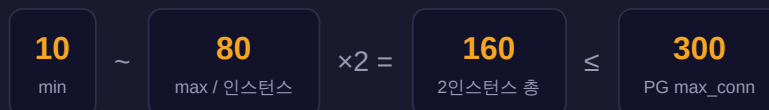
프론트엔드 규약

- 코드 스플리팅 필수: 신규 페이지는 `next/dynamic(() => import(...), { ssr:false })`
- **SWR 캐시**: 참조 데이터(sizes/colors/categories/...)는 `src/hooks/api/` 혹 (5분 dedup)
- **useEffect+apiConnector.get 금지** (참조 데이터) → SWR 혹 교체 필수
- 순차 API 호출 금지 → `Promise.all()`
- Context value 반드시 `useMemo`
- 고트래픽 리스트에 `React.memo` (ProductsList, SalesListView)
- 이미지 `next/Image` · Pagination pageSize 최대 **50** (500 금지)
- AG Grid `ensureAgGridInit()` 1회만 호출

백엔드 규약

- **인메모리 캐시**: 참조 데이터 **60초**, 대시보드 **30초** TTL (MemoryCacheService)
- **slow query**: 100ms 이상 쿼리는 즉시 최적화
- 쿼리 효율로 우선 해결 (pool 확대 전)

PostgreSQL Pool



500명 동시접속 대비. 변경 시 max_connections + pgbouncer 영향 검토 필수.

Docker 규약: 멀티스테이지 빌드 필수 (builder → runner) · 프론트 `npm start`, 백엔드 `node dist/main` · volume mount `./app` 사용 금지 (dev 전용)

8 SWR 훅 목록

src/hooks/api/ — 참조 데이터는 반드시 이 훅들로 조회 (5분 dedup).

훅	엔드포인트	사용처
usePriceTypes	/price-types	ProductsView, BasicDataCard
useCategoriesByStore	/categories/by-store	ProductsView, BasicDataCard
useSizesByStore	/sizes/by-store	ProductsView
useColorsByStore	/colors/by-store	ProductsView
useSubcategoriesByStore	/subcategories/by-store	ProductsView
useSeasonsByStore	/seasons/by-store	BasicDataCard
useOriginsByStore	/origins/by-store	BasicDataCard
useSuppliersByStore	/suppliers/by-store	BasicDataCard
useBranchByStore	/branch/store/{storeId}	SelectorBranch
useTalleresEtapas	/talleres/etapas/all	Talleres 5개 탭
useTalleresVendors	/talleres/vendors/all	Talleres 3개 탭
useTalleresEnvios	/talleres/envios/all	Talleres 3개 탭

9 개발 환경 & 배포

로컬 실행

```
# 전체 (백엔드+프론트+print+zebra)
./dev.sh

# 개별 실행
npm run dev:api      # api-ventago
npm run dev:app      # ventago-app
npm run dev:print    # print-agent
npm run dev:zebra    # zebra-agent

# Push (서브모듈+에이전트 빌드)
./push-both.sh
```

배포 파이프라인

- Jenkins: front-coolsistema (프론트) · api-coolsistema (백엔드)
- GitHub Actions: build-print-agent.yml · build-zebra-agent.yml — 태그 push 시 Win/mac 빌드
- 빌드 실패 시 로그 파일(#NNN.txt) 분석 → 에러 수정 후 push
- 프론트: docker compose build → npm run build (Next.js)
- 백엔드: NestJS SWC 빌드

ESLint 규칙 — Warning도 에러로 처리되어 빌드를 막음

규칙	대처
newline-before-return	return 문 바로 위에 빈 줄 필요
lines-around-comment	주석(//) 바로 위에 빈 줄 필요
no-unused-vars	import한 변수는 반드시 사용
react-hooks/exhaustive-deps	Warning이므로 빌드는 통과하나 주의

npm workspaces 주의: 패키지가 루트 node_modules/ 로 호이스팅됨. webpack alias 시 ./node_modules/... 하드코딩 금지
→ path.dirname(require.resolve('패키지명/package.json')) 사용.

10 최근 주요 개발 이력

CLAUDE.md 기록 기준 타임라인.

2026-03-31

매장 로고 업로드 (MinIO 저장, 사이드바 하단 표시) · 사이드바 리렌더링 방지 (React.memo/useMemo) · **세션 보안 시스템**: 중복 로그인 차단, 디바이스/IP 감지

2026-04-15

Phase 1 성능 최적화 — bundle-analyzer, route timing, Web Vitals, SWR 캐시, 코드 스플리팅, next/font, 스켈레톤

2026-04-16

Phase 2 SWR 마이그레이션 — ProductsView, BasicDataCard, Talleres 5탭, useBranch/useSellers/useClients → SWR 전환 · swcMinify, tree-shaking, React.memo, next/Image, pageSize 500 → 50, AuthContext ↔ BranchContext 분리

2026-04-16

Phase 13 Zebra Agent — branch_agents 테이블, BranchAgent 모델/서비스/게이트웨이 · zebra-agent Electron 앱 (ZPL, TCP 9100, USB, 3종 라벨) · 다중 에이전트 관리 UI, 터미널-에이전트 매핑 · print-agent 다크 테마/USB/서버 URL 고정 · CI 통합

11 운영 서버 직접 접근 규칙

@aiondadotcom/mcp-ssh MCP로 운영 서버(srv803182 / 62.72.7.245) 직접 SSH 접속. 개발 속도를 위해 허용하되 규칙 엄격 준수.

✓ 기본 허용 (확인 없이)

- SELECT · SHOW · EXPLAIN · \d · \dt 조회성 쿼리
- 성능 프로파일링 (pg_stat_*, pg_locks)
- 스키마 검사 (information_schema)
- SSH 조회 (ls · cat · tail · docker logs · docker ps)

⚠ 사용자 확인 필수

- DDL: CREATE · ALTER · DROP · TRUNCATE
- DML 변경: INSERT · UPDATE · DELETE · MERGE
- 관리: GRANT · REVOKE · VACUUM FULL · REINDEX
- 파일 쓰기/수정 · 서비스 제어 (docker restart/stop)
- 파괴적 명령 (rm · shutdown · kill) — 명시적 지시 시에만

중요 — 로컬 dev vs 운영 환경 차이:

- 로컬 dev: PostgreSQL 15 Docker 컨테이너(dbpostgres), docker exec api_ventago ... 경유
- 운영: PostgreSQL 10 호스트 설치(Docker 아님), sudo -u postgres psql 경유, pgbouncer 5432 프록시
- 마이그레이션 SQL 작성 시 PG10/PG15 문법 호환성 주의 (PG10 은 GENERATED AS IDENTITY 등 제한)
- Docker의 postgresql-dbpostgres-1(포트 54322)은 별도 시스템(coolinvoice 등) — Ventago 아님

SSH alias jhkim-server (IdentityFile: ~/.ssh/id_ed25519)

운영 DB ventago · owner coolsistema

운영 매장 CART(3) · coolsistema(6) · genius(8) · ACE(9)

```
# 조회 예
ssh jhkim-server "sudo -u postgres psql -d ventago \
-c 'SELECT id, name FROM stores ORDER BY id;'"
```

12 주의 사항 & 디자인 규약

핵심 주의 사항

- front-end의 **lint 오류에 특히 주의**
- `apiConnector.remove()` 사용 (`.delete()` 아님)
- DB 마이그레이션은 `api-ventago/migrations/` SQL을 운영 Docker에서 직접 실행
- 에이전트 서버 URL은 코드에 고정 (`SERVER_URL`) — 사용자 입력 없음
- 새 테이블 마이그레이션 끝에 `ALTER OWNER TO coolsistema` 필수 (postgres 소유 시 coolsistema 접근 불가 → 500)

디자인 규약 (auto-load skill)

신규 UI/컴포넌트/페이지 작업 시 `Skill("sketch-findings-ace-online")` 자동 참조:

- Theme: **다크 네이비 + 골드 + MP cyan + sandbox=warning gold**
- 영역별 reference: Configuración / Payment Modal QR / Sandbox indicator / Caja virtual wallet / Refund failure UX
- MUI 5 컴포넌트와 1:1 매핑 가능한 클래스명 + React 구현 스케치